

Command and configuration generation

Command and configuration before LLMs

- Plan what you want to do
- Read documentation for syntax on commands and configuration language
- Craft syntax to perform commands or configure systems
- Execute or deploy
- Debug (e.g. StackOverflow)
- Iterate
- Now
 - Plan what you want to do and express it in natural language
 - Have LLM produce command or configuration

PromptOps to automate security auditing

 **GK** < 1 minute ago
@PromptOps iam roles for ec2 instances that are public, print them in csv format: instance-name, ip, iam-role-name

👏 1 🗨️

4 replies

 **PromptOps** APP < 1 minute ago
📖 Found similar questions.

List EC2 instances and their IAM roles Run
Fetches and displays EC2 instances and their associated IAM roles
Created by gk@ctrlstack.com on 2023-06-09 15:08:42
 Last executed on 2023-06-09 15:11:23

 **PromptOps** APP < 1 minute ago
★ Here is a script generated for you!

This script will retrieve the EC2 instances that are publicly exposed and print their instance name, public IP address, and IAM role name in CSV format.

```
import boto3

# Create an EC2 client
ec2 = boto3.client('ec2')
```

[See more](#)

Run Edit Bookmark

 **PromptOps** APP < 1 minute ago
🔍 Execution started by gk@ctrlstack.com
🏁 Execution finished.
📄 Compiling output.
📬 Sending logs over next 1 messages.

 **PromptOps** APP < 1 minute ago

```
sapphire - bastion, 54.68.61.62, sapphire-BastionIAMInstanceProfile

ruby - bastion, 52.36.241.39, ruby-BastionIAMInstanceProfile

neo4j-gitlab-ci, 34.212.23.50, GitLab-CI-Neo4J-InstanceProfile
```

[See more](#)

Increasingly by adversaries

Home / Innovation / Artificial Intelligence

Cybercriminals are using Meta's Llama 2 AI, according to CrowdStrike

Parts of code exploiting Microsoft's PowerShell tool matches the style of output of large language models.



Written by **Tiernan Ray**, Senior Contributing Writer

Feb. 21, 2024 at 2:17 p.m. PT

The group [Scattered Spider](#) made use of Meta's large language model to generate scripts for Microsoft's PowerShell task automation program. The program was used to download login credentials of employees at "a North American financial services victim," according to CrowdStrike.

Command and configuration generation

A tour of uses...

Google dorks

- Google search "language" for hacking searchees
 - Multi-word string search (e.g. "We the people" to search for documents with this phrase)
 - Content restricted searches (`filetype:pdf` for PDF files, `ext:txt` for files with a txt filename extension)
 - Site restricted searches (`site:pdx.edu` for documents on pdx.edu domains, `@twitter` for content on social media platform Twitter)
 - Text restricted searches for URLs containing specific text (`inurl:admin`), pages with titles containing specific text (`intitle:security`), or pages with specific text (`intext:disallow`)
 - Negation operator (e.g. `-filetype:pdf` removes all results that are PDF files)
 - Logical operators (e.g. `(psychology | computer science) & design` for sites that match psychology design or computer science design)
- Have an LLM produce appropriate dorks from natural language

Linux command generation: JSON parsing

- Web APIs returning data in JSON format
 - Example: Certificate Transparency log for cs.pdx.edu
 - <https://crt.sh/?q=cs.pdx.edu&output=json>

```
1 [{"issuer_ca_id":295819,"issuer_name":"C=US, O=Let's Encrypt, CN=E6","common_name":"busdata.cs.pdx.edu",
  "name_value":"busdata.cs.pdx.edu","id":15304902419,"entry_timestamp":"2024-11-11T08:02:11.696",
  "not_before":"2024-11-11T07:03:41","not_after":"2025-02-09T07:03:40",
  "serial_number":"0313aad04ac803b9d36716d632fe94a39a80","result_count":2},{ "issuer_ca_id":295810,
  "issuer_name":"C=US, O=Let's Encrypt, CN=E5","common_name":"ada.cs.pdx.edu","name_value":"ada.cs.pdx.
  edu","id":15316082887,"entry_timestamp":"2024-11-07T08:02:55.486","not_before":"2024-11-07T07:04:24",
  "not_after":"2025-02-05T07:04:23","serial_number":"0358e8567d145308ea02b6d4caba230699ee",
  "result_count":2},{ "issuer_ca_id":295810,"issuer_name":"C=US, O=Let's Encrypt, CN=E5","common_name":"ada.
  cs.pdx.edu","name_value":"ada.cs.pdx.edu","id":15248330149,"entry_timestamp":"2024-11-07T08:02:54.758",
  "not_before":"2024-11-07T07:04:24","not_after":"2025-02-05T07:04:23",
  "serial_number":"0358e8567d145308ea02b6d4caba230699ee","result_count":2},{ "issuer_ca_id":295815,
  "issuer_name":"C=US, O=Let's Encrypt, CN=R11","common_name":"dbclass.cs.pdx.edu","name_value":"dbclass.
  cs.pdx.edu","id":15189233783,"entry_timestamp":"2024-10-30T07:03:17.308",
  "not_before":"2024-10-30T06:04:46","not_after":"2025-01-28T06:04:45",
  "serial_number":"0407ac681e2393a66222d13c23d97ac10aac","result_count":2},{ "issuer_ca_id":295815,
  "issuer_name":"C=US, O=Let's Encrypt, CN=R11","common_name":"dbclass.cs.pdx.edu","name_value":"dbclass.
  cs.pdx.edu","id":15139948601,"entry_timestamp":"2024-10-30T07:03:16.39",
  "not_before":"2024-10-30T06:04:46","not_after":"2025-01-28T06:04:45",
  "serial_number":"0407ac681e2393a66222d13c23d97ac10aac","result_count":2},{ "issuer_ca_id":295815,
  "issuer_name":"C=US, O=Let's Encrypt, CN=R11","common_name":"feedback.cs.pdx.edu","name_value":"feedback.
  cs.pdx.edu","id":15157578867,"entry_timestamp":"2024-10-28T07:01:19.028",
  "not_before":"2024-10-28T06:02:48","not_after":"2025-01-26T06:02:47",
  "serial_number":"044a0bb38d7d76bb94ad573ac9c7fea97d93","result_count":2},{ "issuer_ca_id":295815,
  "issuer_name":"C=US, O=Let's Encrypt, CN=R11","common_name":"feedback.cs.pdx.edu","name_value":"feedback.
```

- Extract information without writing any code

- Method #1: jq
 - Ask an LLM for a jq command to return specific information from a given JSON blob (e.g. all hostnames returned)
 - LLM reverse engineers schema and generates command to access it
 - Run command and verify correctness
- Method #2: Ask LLM directly
 - Supply JSON and ask the same question
 - LLM performs task directly
 - Compare results

iptables

- Commands for filtering network traffic
 - Allow incoming ssh, http, and https

```
iptables -A INPUT -i eth0 -p tcp -m multiport --dports 22,80,443 \
-m state --state NEW,ESTABLISHED -j ACCEPT
```

- Allow outgoing packets from established ssh, http, and https connections

```
iptables -A OUTPUT -o eth0 -p tcp -m multiport --sports 22,80,443 \
-m state --state ESTABLISHED -j ACCEPT
```

- Have LLM explain commands
- Ask an LLM for a prompt that would reproduce them
- Adapt prompt to reproduce commands verbatim

Google Cloud

- gcloud command-line interface for cloud project
- Example for the lab

```
gcloud compute firewall-rules create pdx-80 \  
  --allow=tcp:80 --source-ranges="131.252.0.0/16" \  
  --target-tags=pdx-80 --direction=INGRESS
```

```
gcloud compute instances create wfp1-vm \  
  --machine-type e2-micro --zone us-west1-b \  
  --image-project pdx-cs \  
  --image wfp1-nofilter
```

```
gcloud compute instances add-tags wfp1-vm --tags=pdx-80 --zone  
us-west1-b
```

- Have LLM explain commands
- Ask an LLM for a prompt that would reproduce them
- Adapt prompt to reproduce commands verbatim

nginx configuration

- Popular web server with accompanying configuration specification language

```
server {  
    server_name mashimaro.cs.pdx.edu;  
    listen 80;  
    root /var/www/html/mashimaro;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ =404;  
        autoindex off;  
    }  
  
    add_header Content-Security-Policy"default-src 'none'; script-src  
'self'; connect-src 'self'; img-src 'self'; style-src 'self';" always;  
}
```

- Have LLM explain each line of configuration
- Ask an LLM for a prompt that would reproduce the configuration
- Adapt prompt to reproduce configuration verbatim

Terraform specification

- Terraform Infrastructure as Code solution with accompanying specification language (Hashicorp Configuration Language)

```
provider "google" {  
  credentials = file("tf-lab.json")  
  project     = "YOUR_PROJECT_ID"  
  region      = "us-west1"  
}  
  
resource "google_compute_address" "static"  
{  
  name = "ipv4-address"  
}
```

```
resource "google_compute_instance" "default" {  
  name          = "tf-lab-vm"  
  machine_type  = "e2-medium"  
  zone          = "us-west1-b"  
  
  boot_disk {  
    initialize_params {  
      image = "ubuntu-os-cloud/ubuntu-  
2204-jammy-v20240501"  
    }  
  }  
}
```

```
network_interface {  
  network = "default"  
  access_config {  
    nat_ip =  
google_compute_address.static.address  
  }  
}  
  
output "ip" {  
  value =  
google_compute_instance.default.network_in  
terface.0.access_config.0.nat_ip  
}
```

- Have LLM explain each line of configuration
- Ask an LLM for a prompt that would reproduce the configuration
- Adapt prompt to reproduce configuration verbatim

Docker

- Dockerfile specification

```
FROM python:3.12-alpine3.19 as builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt && \
    rm -rf /usr/local/lib/python3.12/site-packages/*.dist-info
COPY . .
```

```
FROM python:3.12-alpine3.19
WORKDIR /app
COPY --from=builder /app /app
COPY --from=builder /usr/local/lib/python3.12/site-packages/
/usr/local/lib/python3.12/site-packages/
ENTRYPOINT ["python", "app.py"]
```

- Have LLM explain each line of configuration
- Ask an LLM for a prompt that would reproduce the configuration
- Adapt prompt to reproduce configuration verbatim

Kubernetes specification

- Kubernetes Infrastructure as Code solution using YAML (Yet Another Markup Language)

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: guestbook-replicas
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: guestbook
        tier: frontend
    spec:
      containers:
      - name: guestbook-app
        image:
gcr.io/YOUR_PROJECT_ID/gcp_gb
        env:
        - name: PROCESSES
          value: guestbook
        - name: PORT
          value: "8000"
      ports:
      - containerPort: 8000
```

```
apiVersion: v1
kind: Service
metadata:
  name: guestbook-lb
  labels:
    app: guestbook
    tier: frontend
spec:
  type: LoadBalancer
  ports:
  - port: 80
    targetPort: 8000
  selector:
    app: guestbook
    tier: frontend
```

- Have LLM explain each line of configuration
- Ask an LLM for a prompt that would reproduce the configuration
- Adapt prompt to reproduce configuration verbatim

Command agents

Program

- MCP tool implementing natural language interface to Linux CLI

```
@mcp.tool("command")
async def command(command: str):
    """Runs an arbitrary Linux command"""
    loop = asyncio.get_running_loop()
    result = await loop.run_in_executor(
        None,
        lambda: subprocess.run(
            shlex.split(command),
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True
        )
    )
    return {
        "stdout": result.stdout,
        "stderr": result.stderr,
        "returncode": result.returncode,
    }
```


- Client agent with parameterized prompt

```
user_instructions = sys.argv[1]

fast = FastAgent("Command Agent")

@fast.agent(
    instruction = user_instructions,
    model = "gemini25",
    servers = ["command"],
    use_history = True
)
```

Linux command generation: Processes

- Commands and files
 - `ps` (list processes), `ls -l /proc` (list open files per process)
 - `ls /proc` (list objects in file system interface to the kernel)
 - `crontab, /etc/cron.d`
- Examples
 - Using `sudo`, find the names of all processes with open network connections (`ls -l /proc/<PID>/fd`)
 - Show the path of the executable file used to launch `sshd` (`/proc/<PID>/exe`)
 - Show all processes whose original binaries have been deleted (`/proc/<PID>/exe => deleted`)
 - Show the current working directory of the Python process (`/proc/<PID>/cwd`)
 - Find the PID of the Python processes, then use `/proc` to show its environment (`ps, /proc/<PID>/environ`)
 - Show all processes which have the current working directory of `/tmp` (`ps, /proc/<PID>/cwd`)
 - Show all scheduled tasks (`crontab -u, /etc/cron.d`)

Linux command generation: File search, regexp

- Commands
 - `egrep` (text and regular expression file search)
 - `wc` (word/line/character count)
- Examples (searching `rockyou.txt`)
 - How many passwords begin with the letter a (`"^a"`)
 - How many passwords begin with the number 1 (`"^1"`)
 - How many passwords consist of only alphabetic characters (`"^[a-zA-Z]+$"`)
 - How many passwords are made up of exactly 6 numbers (`"^[0-9]{6}"`)
 - List all passwords that begin with `abcde` and end with the number 9 (`"^abcde.*9$"`)

Linux command generation: File system search

- Command
 - `find` (find files in file system)
- Examples
 - Find all `setuid` and `setgid` programs in `/bin` (`-perm 4000, -perm 2000`)
 - Find all ELF executables in `/bin` (`-exec file {}`)
 - Find all files modified in the last day in `/etc` (`-mtime 1`)
 - Find all files created in the last day in `/etc` (`-ctime 1`)
 - Find all `ssh authorized_keys` files in `/home` (`-type f -name`)
 - Find all history files in user directories that have size 0 (`-type f -size 0 -name ".*history"`)

Linux command generation: Network

- Command
 - `iptables` (firewall rule configuration)
- Examples
 - What command allows `ssh` traffic from `131.252.220.0/24`?
 - `-A INPUT -p tcp --dport 22 -s 131.252.220.0/24 -j ACCEPT`
 - What command allows incoming connections to ports 80 and 443 from `131.252.0.0/16`?
 - `-A INPUT -p tcp -m multiport --dports 80,443 -s 131.252.0.0/16 -j ACCEPT`
 - What command allows connections to a MySQL database server at `10.0.0.10`?
 - `-A OUTPUT -p tcp -d 10.0.0.10 --dport 3306 -j ACCEPT`
 - `-A INPUT -p tcp -s 10.0.0.10 --sport 3306 -m conntrack --ctstate ESTABLISHED -j ACCEPT`
 - What command denies all traffic from `1.1.1.1`
 - `-A INPUT -s 1.1.1.1 -j DROP`

Linux command generation: Audit logging

- Commands
 - `who` (list logged in users)
 - `last`, `lastb`, `lastlog` (list prior successful and failed logins)
- Examples
 - Find all currently logged in users (`who`)
 - Find the last 5 successful logins (`last -n 5`)
 - Find the last 5 unsuccessful logins and the IP addresses they came from (`lastb -n 5`)
 - Find the last 5 logins from the root user (`last -n 5 root`)

Linux command generation: Network tracing

- Commands
 - `tcpdump` (capture and analyze network traffic)



- Agent for capturing and analyzing traffic using `tcpdump`
 - What destination ports are being accessed currently?
 - `dig www.pdx.edu`
 - What was the destination of the last web request?
 - `curl https://oregonctf.org`
 - What addresses are communicating over `ssh`?
 - `ssh linux.cs.pdx.edu`

Linux package vulnerabilities

- Automate analysis of vulnerable components in a Linux system's supply-chain
 - `systemctl`
 - Finds running services on system
 - `dpkg-query`
 - Examines installed packages and their versions
 - `apt changelog`
 - Retrieves change logs for a package to see if there are vulnerabilities that have been patched between current version and running version